AlagaAlerta: alertas comunitários de alagamento em São Paulo

Global Solution CitizenTech · Coding for Security · FIAP 2026 · Autor: Nycollas Kaua Pereira Chagas

1. Problema e cidadão

Cidadão afetado: pessoas que moram, trabalham ou circulam perto de áreas de fundo de vale em São Paulo: pedestres, idosos, estudantes, entregadores, motoristas por aplicativo e pequenos comerciantes. **Problema:** em chuvas fortes o alagamento se forma rápido e rua a rua, mas a informação sobre quais pontos estão intransitáveis circula de forma fragmentada, atrasada e sem padrão (grupos de mensagem, redes sociais). Alertas oficiais costumam cobrir regiões amplas, não o ponto exato do trajeto. O resultado é gente entrando em vias alagadas, horas perdidas e risco à vida. A solução combina duas ideias da etapa de Inspiração: *previsão de enchentes com alertas direcionados* e *zeladoria colaborativa*: cidadãos e sensores relatam pontos alagados e o sistema os cruza com a chuva prevista.

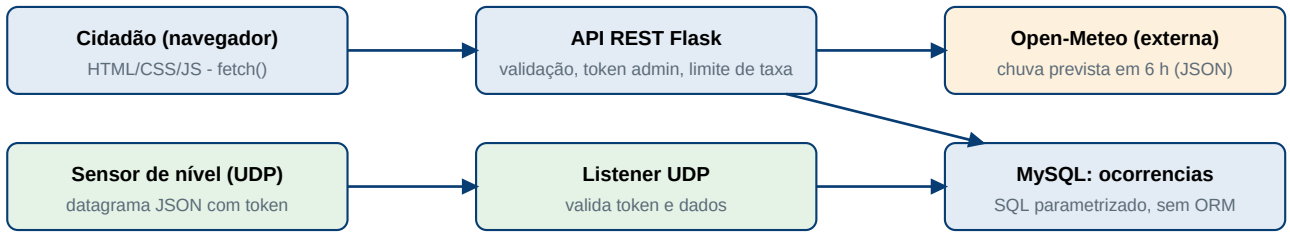
2. Evidências e hipóteses

- **Observação do cenário:** alagamentos em vias e córregos são recorrentes nos verões da capital paulista e atingem o deslocamento diário.
- **Dado público:** a previsão horária de precipitação da API aberta Open-Meteo, consumida pelo protótipo, é convertida em risco por bairro (soma das próximas 6 h).
- **Hipóteses a validar com usuários:** (H1) moradores reportam pontos se o relato for rápido e anônimo; (H2) previsão somada a relatos reais é mais útil que qualquer um dos dois isolados.

3. ODS relacionados

ODS	Como a solução contribui
11 Cidades e comunidades sustentáveis	Meta 11.5: reduzir mortes, pessoas afetadas e perdas por desastres relacionados à água, com informação por ponto da cidade.
13 Ação contra a mudança global do clima	Meta 13.1: fortalecer a resiliência a riscos climáticos; eventos extremos de chuva exigem alerta local e rápido.
9 Indústria, inovação e infraestrutura	Infraestrutura digital resiliente e de baixo custo; sensores simples via UDP funcionam com conectividade limitada.

4. Arquitetura e modelo de dados



Log em arquivo (data/hora, ação, resultado) gravado pela API e pelo listener; sem dados pessoais.

Tecnologias: Python 3, Flask, MySQL com mysql-connector-python (SQL puro, sem ORM), requests (Open-Meteo), sockets UDP, HTML/CSS/JS sem frameworks, logging com arquivo rotativo. Código dividido em módulos: `routes, db, validators, weather, security, logger, udp_listener`.

Endpoint	Função	Status
GET /api/ocorrencias (e /<id>)	Lista com filtros ou detalha uma ocorrência	200, 400, 404
POST /api/ocorrencias	Cria relato do cidadão (limite de taxa)	201, 400, 413, 429
PUT /api/ocorrencias/<id>	Atualiza nível, descrição ou status (token admin)	200, 400, 401, 404
DELETE /api/ocorrencias/<id>	Remove ocorrência (token admin)	204, 401, 404
GET /api/risco?bairro=	Chuva prevista em 6 h e nível de risco (fonte externa)	200, 400, 503

Tabela ocorrencias	Tipo	Observação
id	INT UNSIGNED, PK, auto incremento	Identificador
bairro / referencia	VARCHAR(40) / VARCHAR(120)	Bairro de lista fixa; ponto de referência (sem endereço residencial exato)
nivel	TINYINT, CHECK 1 a 4	1 atenção, 2 via alagada, 3 intransitável, 4 risco grave
descricao	VARCHAR(280), nulo	Texto livre curto, sem os símbolos < e >
status / origem	ENUM('ativa','resolvida') / ENUM('cidadao','sensor')	Origem é definida pelo servidor, nunca pelo cliente
criado_em / atualizado_em	DATETIME	Preenchidos pelo banco; índice por (bairro, status)

5. Segurança e privacidade

LGPD e coleta mínima: o sistema não grava nome, telefone, e-mail nem IP; o relato descreve um local público, não uma pessoa. O log registra data/hora, ação e resultado, sem corpo de requisição, tokens ou IPs (e neutraliza quebras de linha

para evitar *log injection*). Todos os dados do protótipo são fictícios ou abertos. **Controles gerais:** credenciais em `.env` (fora do Git, tokens com mínimo de 16 caracteres), consultas parametrizadas, validação por allowlist de toda entrada, erros genéricos ao cliente (detalhes só no log), usuário do banco com privilégios mínimos e cabeçalhos de segurança (CSP, nosniff).

Ameaça	Como ocorreria	Mitigação implementada
1. Injeção (SQL) e XSS armazenado	Texto malicioso em referencia/descricao ou nos filtros da URL alteraria consultas ou executaria script no navegador de outros usuários.	SQL sempre com %s e colunas de UPDATE em allowlist; validação por regex e tamanho (rejeita < >); interface usa textContent (nunca innerHTML) e CSP; testes com payloads.
2. Abuso e dados falsos (spam, falsificação de sensores, adulteração)	Robô enviando milhares de relatos; pacote UDP forjado criando alertas falsos; terceiro apagando ocorrências.	Limite de taxa no POST (429); corpo máx. 8 KB; PUT/DELETE só com token admin comparado em tempo constante; origem fixada pelo servidor; leitura UDP só com token, até 512 bytes e validada.
3. Exposição de credenciais e de detalhes internos	Senha do banco no repositório ou mensagem de erro revelando SQL, caminhos ou versões ajudaria um atacante.	Segredos só no .env (.gitignore + .env.example sem valores reais); app recusa iniciar com segredo fraco; handler global devolve erro genérico e grava detalhe no log; debug desligado.

6. Validação e viabilidade

- **Testes automatizados (33, todos aprovados):** CRUD com status corretos; 400 para 11 tipos de entrada inválida; payloads de SQL injection e XSS rejeitados; 401 sem token; 429 no limite de taxa; 500 sem vazar detalhes; 503 quando a fonte externa falha; log com data/hora, ação e resultado; camada db.py verificada com conexão simulada (nenhum valor do usuário entra no texto do SQL); socket UDP real em localhost.
- **Integração real:** MySQL e Open-Meteo são exercitados pelo roteiro do README (curl e interface), pois os testes usam banco simulado.
- **Viabilidade:** custo baixo (software livre, Open-Meteo gratuita para uso não comercial, sensor simples com microcontrolador). Operação natural por parceria com Defesa Civil/prefeituras ou ONGs de bairro; o mesmo modelo escala trocando a lista de bairros.

7. Limitações e próximos passos

- Hipóteses H1 e H2 ainda não foram testadas com cidadãos; é preciso um piloto de usabilidade em um bairro.
- Os limiares de risco (10/25/50 mm em 6 h) são uma heurística didática, não critérios oficiais; devem ser calibrados com dados da Defesa Civil.
- Sem autenticação de cidadãos, relatos falsos são possíveis (mitigados por limite de taxa e moderação); falta HTTPS (proxy reverso), criptografia no UDP (VPN/DTLS), limitador de taxa compartilhado entre processos e expiração automática de relatos antigos.
- Lista fixa de 10 bairros com coordenadas aproximadas do centro; ampliar para busca por ponto no mapa.

8. Recursos além do conteúdo visto (bônus) e por que foram usados

Limitação de taxa em memória (deque + lock): contém spam sem depender de serviços externos. **hmac.compare_digest**: compara tokens sem vazar informação por tempo de resposta. **Content-Security-Policy e demais cabeçalhos**: segunda barreira contra XSS. **unittest.mock**: permite testar banco, API externa e falhas sem infraestrutura. **threading.Event no listener UDP**: encerramento limpo e teste com socket real. **Cache com TTL**: respeita a fonte externa e mantém a página rápida. **RotatingFileHandler**: impede o log de crescer sem limite.